End-to-End Saliency Object Detection Pipeline

Architecture Transition from Baseline CNN to U-Net

AI Engineering Intern

Genpact GigaAcademy Program

University for Business and Technology (UBT)

May 8, 2026

Abstract

This report details the development of an end-to-end Machine Learning pipeline for Saliency Object Detection (SOD), completed as part of the Genpact GigaAcademy AI Engineering Internship. The project objective was to design, train, and deploy a deep learning model capable of accurately identifying and extracting primary subjects from images. The project involved transitioning from a baseline Convolutional Neural Network (CNN) to an advanced U-Net architecture. By incorporating skip connections, Batch Normalization, Dropout, and a custom Hybrid Loss function, the improved model demonstrated significant performance gains. Final evaluation on the MSRA10K test set yielded a Mean IoU of 0.6720 and an F1-Score of 0.8030. Furthermore, the model was successfully deployed as an interactive, real-time web application using Gradio, achieving an inference latency of approximately 182 ms per image.

1. Introduction

Saliency Object Detection (SOD) is a critical computer vision task that aims to identify the most visually conspicuous objects in an image and separate them from the background. Unlike standard semantic segmentation, which categorizes every pixel into predefined classes, SOD focuses on binary classification: distinguishing the "subject" (foreground) from the "non-subject" (background).

This capability is essential for downstream tasks such as automatic background removal, visual tracking, and image editing. During the Genpact AI Engineering Internship, the goal was to build a complete software engineering and machine learning pipeline for this task. The project encompassed data ingestion, preprocessing, custom model architecture design using PyTorch, rigorous evaluation, and deployment via an interactive web interface.

2. Dataset and Preprocessing Pipeline

2.1 Data Source

The project utilized the MSRA10K dataset, a standard benchmark in saliency detection containing 10,000 images paired with pixel-level binary masks. To rigorously validate the architecture, the dataset was split into three strict subsets: 70% for Training, 15% for Validation, and 15% for Testing.

2.2 Preprocessing and Augmentation

To ensure computational efficiency and uniform tensor dimensions, all input images and corresponding ground-truth masks were resized to 128×128 pixels. To improve model generalization and prevent overfitting on the training set, a robust data augmentation pipeline was implemented using the `torchvision.transforms` library:

- **Horizontal Flipping:** Applied with a 50% probability, strictly synchronized between the image and mask to prevent misalignment.
- **Random Resized Cropping:** Extracted a random patch (80% to 100% of the image area) and resized it back to the target 128×128 resolution.
- **Brightness Adjustment:** Random brightness jittering (factor 0.8 to 1.2) applied exclusively to the input RGB image, leaving the mask unaffected.

Images were normalized to a $[0, 1]$ floating-point tensor range, and masks were strictly thresholded at 0.5 to maintain binary integrity.

3. Model Architectures

The project followed an iterative development approach, establishing a baseline before evolving into a state-of-the-art configuration.

3.1 Baseline Convolutional Neural Network

The initial model was a symmetric Encoder-Decoder CNN constructed from scratch.

- **Encoder:** Consisted of four `Conv2d` layers, each followed by a ReLU activation and a 2×2 Max Pooling layer. This block progressively reduced spatial dimensions while increasing channel depth to extract high-level semantic features.
- **Decoder:** Utilized four `ConvTranspose2d` layers to upsample the latent representation back to the original resolution, concluding with a Sigmoid activation.

While the baseline model successfully identified general object locations, it suffered from boundary degradation (blocky, pixelated edges) and was prone to false positives in heavily textured backgrounds.

3.2 Improved Architecture: The U-Net

To address the limitations of the baseline CNN, the architecture was upgraded to a custom U-Net configuration. The U-Net architecture inherently solves spatial resolution loss through the introduction of **Skip Connections**.

By concatenating the high-resolution feature maps from the encoder directly with the upsampled feature maps in the decoder, the network retains precise edge details. Additionally, the improved model introduced:

- **Batch Normalization:** Applied after convolutional layers to stabilize gradient flow and accelerate training convergence.
- **Dropout Layers:** Integrated into the bottleneck and decoder stages (with $p = 0.5$ and $p = 0.3$) to severely penalize co-adaptation of neurons, acting as a strong regularizer against background noise.

4. Training Methodology

4.1 Custom Hybrid Loss Function

Saliency detection tasks frequently suffer from class imbalance (background pixels heavily outnumber foreground pixels). To combat this, a custom hybrid loss function was implemented, combining standard Binary Cross-Entropy (BCE) with the Intersection over Union (IoU) metric.

$$L_{total} = BCE(P, G) + 0.5 \times (1 - IoU(P, G))$$

Where the Jaccard Index (IoU) is defined as:

$$IoU(P, G) = \Sigma(P \times G) / (\Sigma P + \Sigma G - \Sigma(P \times G) + \varepsilon)$$

Here, P represents the predicted probability map, and G represents the Ground Truth binary mask. The inclusion of the IoU penalty directly optimizes the network for structural overlap, penalizing false positives and false negatives equally.

4.2 Optimization Setup

- **Optimizer:** Adam Optimizer with an initial learning rate of 1×10^{-3}.
- **Scheduler:** A learning rate scheduler reduced the LR by a factor of 0.5 if the validation loss stagnated for 2 consecutive epochs.
- **Early Stopping:** Implemented to halt training if validation metrics failed to improve over 3 epochs (patience), preventing the U-Net from overfitting the training data.

5. Evaluation and Results

The models were evaluated exclusively on the held-out Test Set (15% of the data) using classification and regression metrics provided by `scikit-learn`.

5.1 Quantitative Metrics

The transition to the U-Net architecture yielded substantial quantitative improvements across all tracked metrics, particularly in the precision of the generated masks.

Evaluation Metric	Baseline CNN	Improved U-Net	Improvement
Mean IoU (mIoU)	0.5852	**0.6720**	+ 14.8%
F1-Score	0.7374	**0.8030**	+ 8.8%
Precision	0.7512	**0.8245**	+ 9.7%
Recall	0.7241	**0.7826**	+ 8.0%
Mean Absolute Error	0.1250	**0.0890**	- 28.8%

5.2 Qualitative Observations

Visual inspection of the output masks confirmed the numerical data. The U-Net efficiently traced complex geometric boundaries (e.g., clothing folds, intricate shapes, hair details) that the baseline CNN smoothed over. Background artifacts, which were prevalent in the baseline model, were largely eliminated due to the regularizing effect of the Dropout layers and the custom Hybrid Loss.

6. Deployment and Engineering Handover

A core requirement of the Genpact internship was demonstrating production readiness. The finalized U-Net weights were integrated into a modular `app.py` script utilizing the Gradio framework and OpenCV.

The deployed web application allows users to upload custom imagery and returns the following four visualization metrics:

- **Input Resolution:** The raw input image scaled to the network's ***128 × 128*** requirement.
- **Binary Mask:** The pure black-and-white saliency mask prediction.

- **Overlayed Visualization:** A heatmap overlay utilizing OpenCV's `addWeighted` function to visually map the mask onto the original image.
- **Latency Tracking:** Real-time inference speed analysis, clocking an average of **182.56 ms** per image on standard local hardware, verifying the model's suitability for real-time processing pipelines.

7. Conclusion and Future Work

This project successfully demonstrated the end-to-end lifecycle of an AI engineering initiative. Transitioning from a theoretical baseline to a deployed, optimized U-Net underscored several key learnings:

- **Architectural Impact:** Spatial resolution loss in deep networks is a severe bottleneck for pixel-level tasks; skip connections are non-negotiable for high-fidelity masking.
- **Loss Dynamics:** Relying solely on BCE is insufficient for imbalanced masks. Directly optimizing for the target metric (IoU) guides the network more efficiently.
- **Software Engineering Discipline:** Modularizing Jupyter Notebook code into object-oriented Python scripts and tracking metrics transparently is as critical as the mathematical architecture itself.

The finalized pipeline establishes a robust foundation for future enhancements. Subsequent iterations of this project could involve integrating pre-trained feature extractors (e.g., ResNet50) as the encoder backbone to improve mIoU further, or packaging the Gradio application into a Docker container for cloud-based deployment across Genpact's infrastructure.